Both forms have identical output, but the preferred form today would be more concise `...$string`. Indeed, the feature was added to the language for precisely this reason.

Incidentally, the change in behavior of the `...` operator demonstrated above is an example of a general programming language feature called *operator overloading*. This is a feature that allows operators to do different things in different contexts. In our case `...` is building an array when used in an argument declaration and decomposing an array into individual elements when used on arrays passed into a function at runtime. (For the curious: no, it isn't possible to change operator behavior like this in your own PHP code; it was discussed in an RFC for PHP 8, but the proposal was declined.)

Named Parameters

PHP 8 introduces a nifty way to pass data into functions using the parameter names. Up to now the order in which we passed variables into our functions has had to align with the order or arguments. Named arguments allows you to label each item of incoming data and PHP will assign the data to its corresponding function argument. Let's learn about classic albums, and also some PHP:

```
function knowYourClassicMetal(string $album, string $artist, string $year,
string $label): void
{
    print "{$album} by {$artist}, released in {$year} on {$label}" .
    PHP_EOL;
}

knowYourClassicMetal(album: 'Somewhere in Time', artist: 'Iron Maiden',
year: '1985', label: 'EMI');
knowYourClassicMetal(artist: 'Slayer', year: '1986', label: 'Def Jam',
album: 'Reign in Blood');

Output:
Somewhere in Time by Iron Maiden, released in 1985 on EMI
Reign in Blood by Slayer, released in 1986 on Def Jam
```